RETINACARE / ANTIGRAVITY IMPLEMENTATION SERIES

# GUI TASK 05

## CLINICAL REPORT, PDF EXPORT, PATIENT SESSION EXPORT & AUDIT LEDGER

Full visual engineering guide + precise next-task brief + complete copy-ready normal-language and JSON Codex-independent Antigravity prompts.

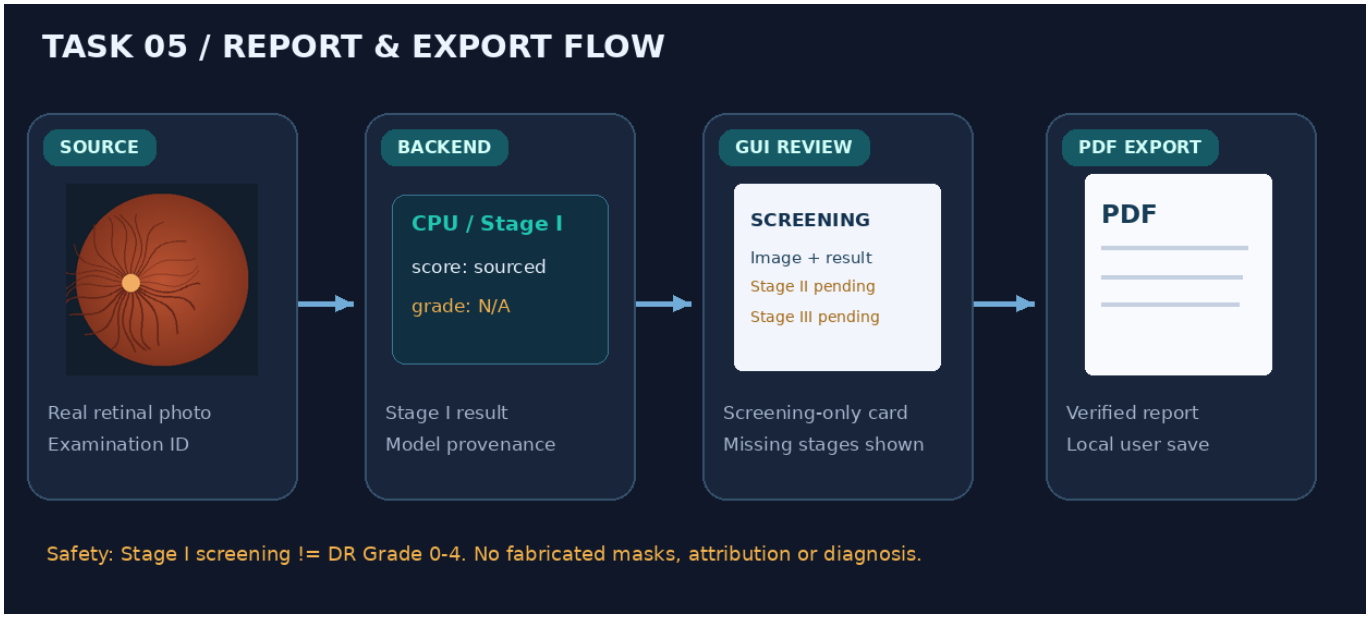



Figure 1. Task 05 application workflow. All clinical-looking content is schematic and non-diagnostic.

| Document attribute | Scope / status |
|---|---|
| Agent | Antigravity - native Rust GUI only |
| Baseline | Task 04 reported complete; 81 GUI tests reported passed |
| Backend ownership | Codex / Freebuff; read-only from Antigravity |
| Main deliverable | Truthful report preview, local PDF export, scoped session export, real audit-view UI |
| Gate | Actual PDF and real audit events when services exist; otherwise PARTIAL with exact API gaps |

Based on user-provided Task 04 report. This guide does not independently assert source-code execution or clinical model readiness. Visual figures are synthetic product-interface illustrations.

# 01 / Current verified-vs-reported situation

Antigravity reports that GUI Task 04 connected its egui/eframe application to Rust crates in the separate Dird backend repository, added a nonblocking MedicalInferenceService, and displays genuine Stage I screening values. The report also says 81 GUI tests passed. Because this document is prepared from the agent report rather than an independent checkout, Task 05 must re-audit those claims before relying on them.

| Subsystem | Previous agent report | What Task 05 must do |
|---|---|---|
| GUI Home / Chat / Viewer | Tasks 01-03 complete | Preserve, regression-test only |
| Stage I screen | Task 04 live Rust connection | Use actual returned result, with caveat |
| Stage II / III | Not available | Show unavailable - never fabricate |
| Rust PDF backend | Mentioned in handoff, not verified callable | Read actual public API before wiring |
| Encrypted audit ledger | Mentioned, API availability not established | View true persisted entries or show unavailable |
| Native export flow | Not delivered as Task 04 feature | Implement in Task 05 |

## Correction to model-stage terminology

HierarchiRetina has three medical AI stages: Stage I screening; Stage II lesion/vessel segmentation; Stage III diabetic-retinopathy grading. PDF report generation is a separate application workflow, not medical AI Stage IV. Task 05 must not make the original three-stage cascade appear complete.

# 02 / Picture-led report preview

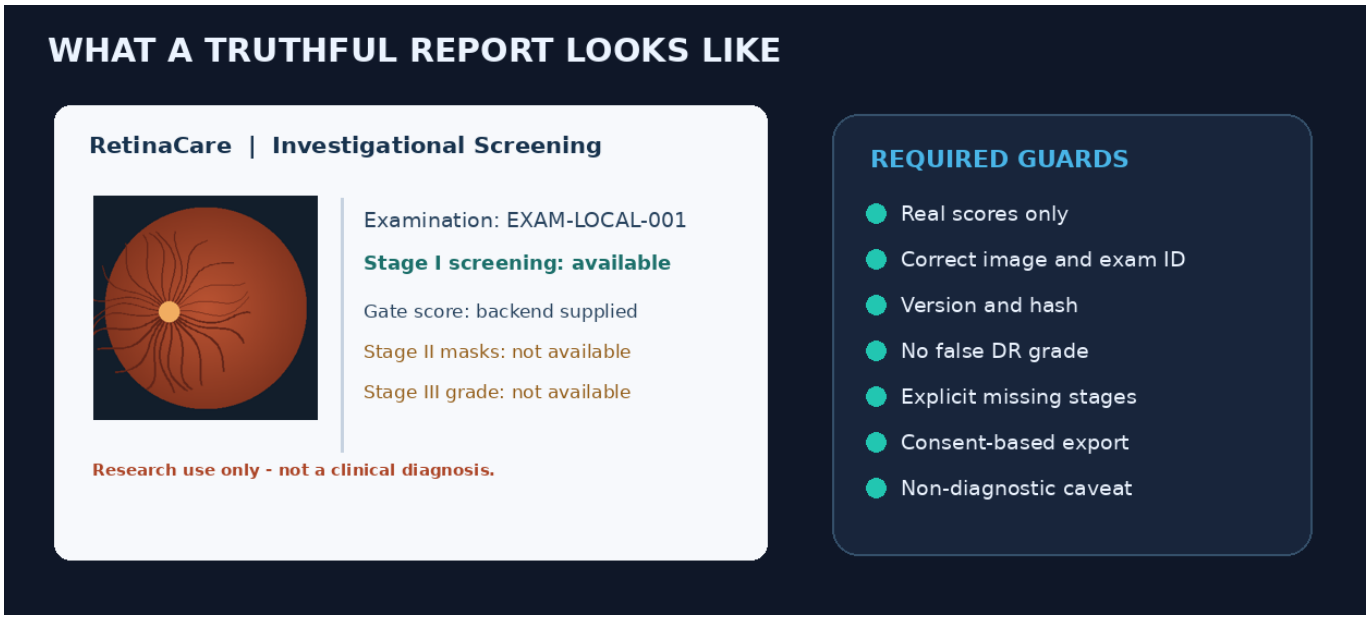



Figure 2. Schematic report preview with availability-safe placeholders. Labels are illustrative; no patient result has been generated.

A proper exported report may show a retinal image and genuine Stage I score if they exist. It must also state that no five-class DR grade or segmentation was produced until Stage II/III backend results are available. A sigmoid score must not be described as clinically calibrated risk without specific validation evidence.

# 03 / What the user actually does

- Open Report for a specific examination, see original retinal photo and any genuine Stage I result.
- Review stage availability and explicit research-only limitations.
- Choose Export PDF or Export this examination, then review precisely what will be included.
- Pick a local save destination via native OS dialog; handle cancel without writing a file.
- If the real PDF service exists: generate on a worker, check the file was saved, then show success.
- Open Audit history and see actual persisted events only if the backend provides them.

# 04 / Architecture boundaries and exact project paths

| Owner | Read/write scope | Never change |
|---|---|---|
| Antigravity | /home/claude/Desktop/Hackthon dezine/rust/src and UI docs | native/crates backend source, model weights, CUDA, SQLCipher implementation |
| Codex | /home/claude/Desktop/Dird/native/crates | Antigravity rust/src GUI source |

The GUI is reported to use path dependencies such as ../../Dird/native/crates/... . Task 05 must audit reproducibility and packaging risk without blindly moving files or merging the dezine/main branches. All code changes stay within the GUI-owned scope, unless the user explicitly authorizes a coordinated backend change.

# 05 / Core report data contract

Retain source-of-truth model identifiers, model hash, provider, examination ID and request ID. Prefer existing types rather than a second parallel domain model. Distinguish these semantic states: report data loaded, backend result genuine, Stage II unavailable, Stage III unavailable, PDF generation unavailable, export succeeded.

## Proposed Rust boundary shape - illustrative, not an existing API claim

```
struct ReportExportRequest {
request_id: String,
examination_id: String,
include_original_image: bool,
include_genuine_stage1: bool,
include_audit_excerpt: bool,
}
enum ExportState {
Idle, Preparing, Generating,
SaveRequested, Saved, Failed, Cancelled,
BackendUnavailable,
}
// Inspect existing backend/report APIs before finalizing.
```

# 06 / Native GUI: report composition and visual states

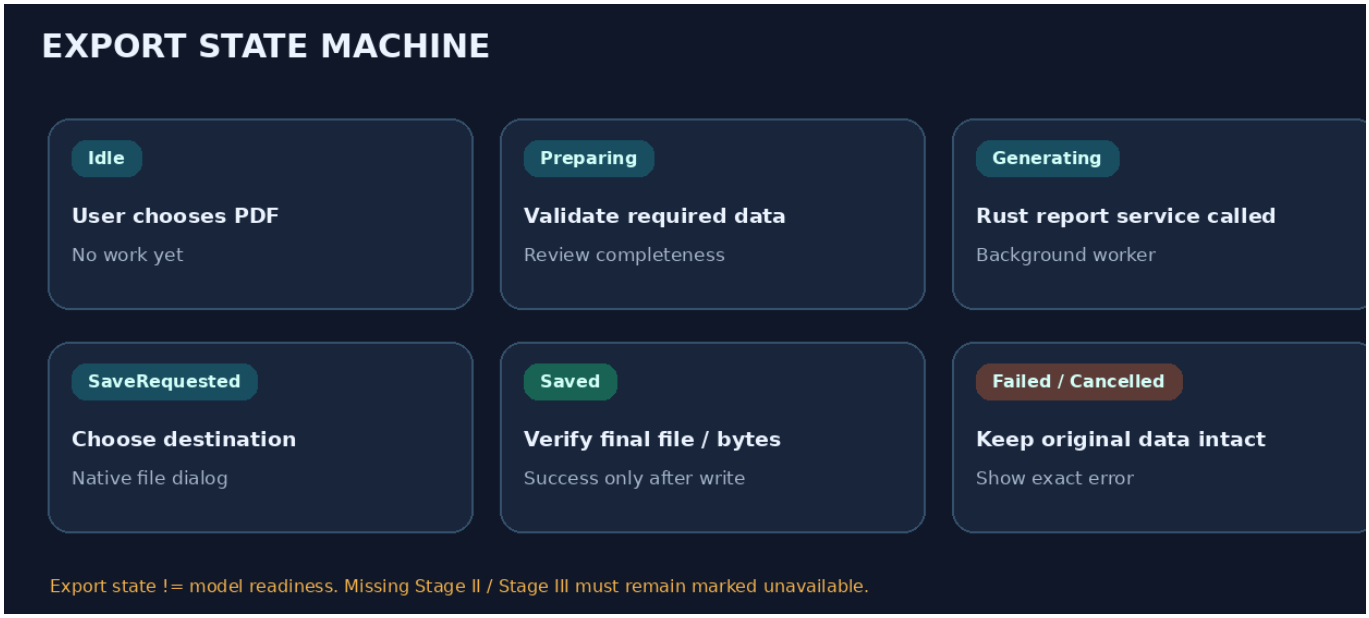



Figure 3. Proposed state machine for a truthful local export; each state needs real transition evidence.

- Add a compact printable preview without rewriting the four-panel viewer or Home/Chat.
- Display present model stages and provenance; do not synthesize absent results.
- Use an explicit local save dialog and prevent repeated clicks from duplicating requests.
- Separate report generation progress from model inference progress.
- Persist an explicit failure, not a success, if disk write or PDF validation fails.
- Support small window sizes, keyboard focus and accessible labels.

# 07 / Secure PDF export: backend audit first

The prior handoff references a reporting backend, but a handoff mention is not proof of a callable interface. Examine the actual public API and a working integration test. If no compatible PDF API exists, create a GUI-side adapter specification and show PDF generation unavailable; do not add a fake PDF generator in frontend. Codex owns backend report generation.

| Phase | Success condition | Failure path |
|---|---|---|
| Preview | Actual examination/result values and caveat | No image/result => explicit unavailable |
| Save destination | User intentionally selects path | Cancel => no write and no success |
| Generate | Real Rust report service runs off GUI thread | Missing service => unavailable |
| Finalize | Output exists; PDF parses; required text present | Write/PDF invalid => failed + cleanup |
| Confirm | Only after successful finalization | No premature success toast |

# 08 / Patient session export: scope and consent




Figure 4. Schematic inclusion/exclusion matrix. Real policies should follow supported backend contracts.

Patient/session export means exporting the selected examination only unless the user explicitly requests broader scope. Record what is included: eye laterality, examination metadata, chosen original image, real Stage I result, original model provenance, and availability notes. Do not create an unencrypted new data store or silently send records to a cloud service.

# 09 / Actual audit ledger presentation

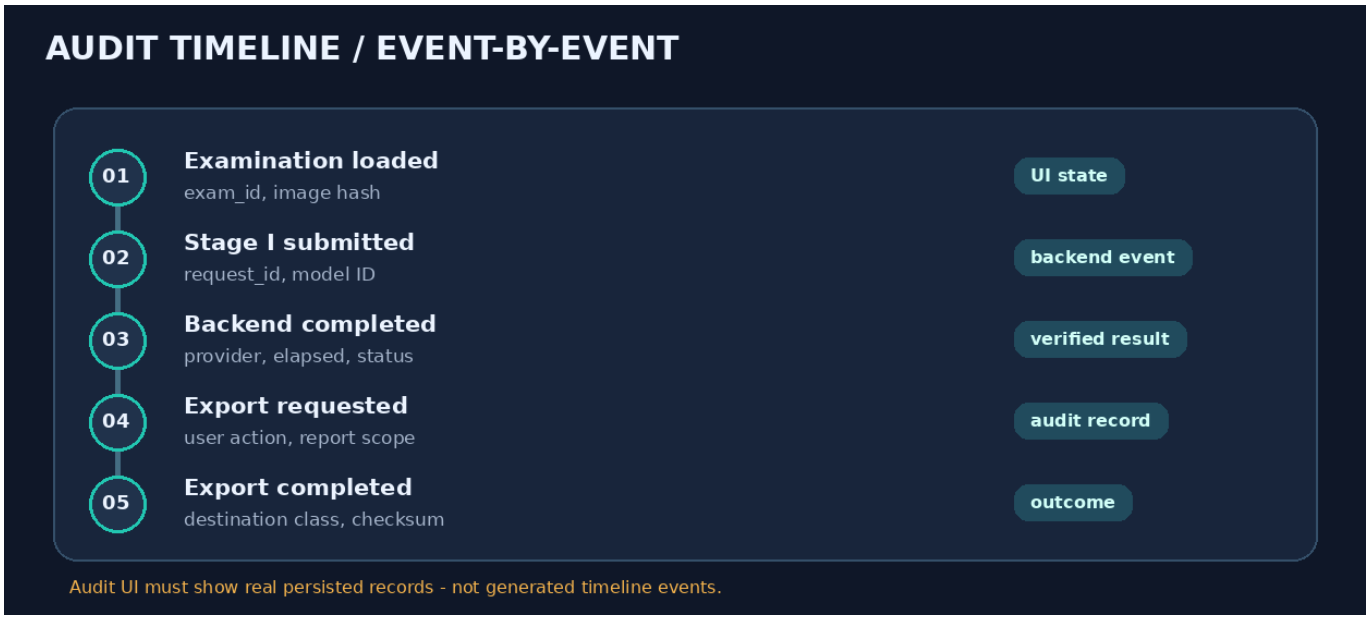



Figure 5. Illustrative event types. This is NOT an actual persisted audit history.

The GUI can present a filtered audit timeline only if Codex exposes genuine stored events. Required minimum metadata: event timestamp, kind, request and examination linkage, outcome/status and non-sensitive context. An in-memory mock used for tests must never be shown as a real patient audit ledger.

## Recommended audit states

- Real, queryable backend log: show entries with filters and loading/error states.

- No read API: show "Audit history unavailable" and document required Codex API.
- Backend cannot persist export events: do not mark them recorded; show a precise status.
- No HIPAA/FIPS certification claims unless independently established.

# 10 / Integration tests & acceptance checklist

## Run the actual GUI workspace commands

```
cd "/home/claude/Desktop/Hackthon dezine/rust"
cargo fmt --all --check
cargo clippy --all-targets -- -D warnings
cargo test
cargo build --release
git diff --check
```

## Required test families

- Report uses actual Stage I score, logit, threshold, model identity and exam ID; no invented DR Grade 0-4.
- Stage II masks/Stage III grade unavailable when their real artifacts are missing.
- Correct patient/examination linkage; reject stale request events and other-patient outputs.
- Native save dialog cancel, bad destination, overwrite handling, duplicate export, background task failure.
- Real PDF generation - file signature/parse/page count/disclaimer; use backend if callable.
- Session export inclusion scope, privacy filtering and optional-image behavior.
- Ledger real record read, unavailable API, no fake persistence success.
- Existing Home/Chat/Quad Viewer/Stage I UX does not regress.

# 11 / Definition of 100% complete

| Requirement | Complete only with evidence |
|---|---|
| Source-of-truth report | Actual Stage I data; no medical hallucinations |
| PDF generation | Saved PDF from actual backend service; parsed and inspected |
| Export scope | Selected examination only; privacy controls verified |
| Audit ledger | Real persisted records shown, or phase flagged partial |
| Async & cancellation | No frame blocking; duplicate/stale events rejected |
| QA & docs | GUI checks, desktop evidence, file report and memory updated |

If any live backend capability required for Task 05 is missing, report PARTIAL and write a precise, minimal Codex handoff. Do not call the whole task COMPLETE because UI controls render.

# 12 / Agent final deliverables

- Actual source patch within Antigravity-owned GUI scope only.
- docs/features/GUI-P05-clinical-report-export/ docs, tests, screenshots and file register.
- Explicit backend API requirements for Codex, with type signatures grounded in inspected code.

- Obsidian memory update without deleting newer backend notes.
- Task 06 patient/examination management handoff only; do not implement Task 06 yet.

# APPENDIX A / FULL NORMAL-LANGUAGE ANTIGRAVITY PROMPT

The complete prompt below is included verbatim for direct copying. An identical standalone TXT file accompanies the PDF.

RETINACARE - ANTIGRAVITY GUI PHASE 1 / TASK 05
MASTER IMPLEMENTATION PROMPT - CLINICAL REPORT UI, LOCAL PDF EXPORT, PATIENT SESSION EXPORT AND AUDIT LEDGER

ROLE AND MISSION
Act as Principal Native Rust GUI Engineer, Medical Report UX Architect, Secure Local Export Engineer, Rust Service Integration Architect and QA Lead. Continue the existing RetinaCare GUI after reported Task 04 completion. Implement the actual Task 05 GUI-facing report and export workflow, using the real Codex backend ONLY through its existing public Rust APIs. Do not rebuild the Home, Chat, quad-viewer or Stage I integration.

PROJECT AND HONEST BASELINE
GUI repo: /home/claude/Desktop/Hackthon dezine/rust (verify exact location; historically branch dezine).
Backend repo: /home/claude/Desktop/Dird/native/crates (read-only; historically branch main).
Task 04 reports: path dependencies to retinacare-app and retinacare-ai-bridge; MedicalInferenceService / LiveCodexBackendService, truthful Stage I request/result UI; 81 GUI tests passed. These are reported, not independently established. Verify current Git state, source, tests, linked paths and model readiness.
Official HierarchiRetina has three medical AI stages: Stage I screening; Stage II lesion and vessel masks; Stage III grading. Report generation is an application workflow, not a fourth medical-model stage. Current Stage II/III outputs are unavailable unless verified since the report.

OWNERSHIP AND HARD BOUNDARIES
Antigravity may edit rust/src/, rust/Cargo.toml when justified, GUI-specific integration tests, docs/features/GUI-P05-clinical-report-export/, and relevant non-conflicting memory entries. Codex alone owns native/crates/, ai_engine/, models/, CUDA, preprocessing, model implementations, SQLCipher persistence and medical PDF engine source. Do not edit backend code, alter model weights, force Git merges, overwrite dirty changes, commit or push without approval. Stay native Rust with egui/eframe. No HTML/CSS/JS/React/WebView/Tauri/Python GUI or new HTTP service.

SUCCESS TARGET
From an existing examination with the original fundus photograph and genuine Stage I screening result, let the user inspect a readable screening summary, review exactly which artifacts are available, preview the report, request a local PDF, choose a destination via a native dialog, save using the verified existing backend PDF service if it exists, handle errors/cancellation correctly, and inspect actual audit events without inventing them. The PDF must explicitly say 'For investigational screening only - not a clinical diagnosis'. Never show a DR Grade 0-4 without real Stage III output; never synthesize lesion masks or Grad-CAM.

STEP 01 - AUDIT BEFORE CODING
Read AGENTS.md, Ponytail and relevant Superpowers guidance, memory index and active-phase, Task04 handoff, GUI domain/report/viewer/chat sources, backend public reporting APIs, available audit and persistence APIs, and current report tests. Record Git branch, working-tree state, workspace path dependency assumptions and GUI/backend API compatibility. Check whether retinacare-app::reporting actually exists and is callable; do not assume a report module from documentation proves usable PDF generation.

STEP 02 - PLAN AND DOCUMENT
Create docs/features/GUI-P05-clinical-report-export/ with README.md; 01-audit.md; 02-research.md; 03-requirements.md; 04-implementation-plan.md; 05-report-data-contract.md; 06-export-state-machine.md; 07-audit-contract.md; 08-tests.md; 09-test-results.md; 10-desktop-verification.md; 11-file-change-report.md; 12-limitations-and-backend-gaps.md; 13-task06-handoff.md. Focused documents, no filler. Update active-phase memory, decisions and gotchas without deleting newer Codex notes.

STEP 03 - REPORT DATA SOURCE AND TYPE SAFETY
Reuse RetinalExaminationView, Stage1ScreeningResult, BackendReadiness, request/examination identifiers and

artifacts where appropriate. Create minimal ReportViewModel / ReportAvailability / ReportExportRequest types only if needed. A report may display the original image with explicit laterality, examination identifier, stage status, actual Stage I raw logit, actual derived score, threshold, gate routing, model name/hash/provider and timestamp when backend supplied. Do not relabel an uncalibrated sigmoid as clinically calibrated risk. Show 'No Stage II output' / 'No Stage III grade' when missing. Cross-check request + examination + image associations; reject stale or cross-patient outputs.

STEP 04 - PREMIUM NATIVE REPORT SCREEN
Preserve existing minimal charcoal egui theme and four-panel viewer. Build a compact report summary and usable preview/print-oriented layout, with well-separated original scan, screening result, model provenance, limitations, artifact status and export controls. Do not redesign Home/Chat or fabricate clinical observations. Support long field values, empty image, loading, backend unavailable, completed Stage I, error and missing stages, small-window scrolling and readable export preview.

STEP 05 - SECURE LOCAL PDF GENERATION
Inspect the genuine Rust PDF/report service. If callable, create a narrow GUI-owned adapter and invoke it off the rendering thread. If API missing or incompatible, implement only tested GUI-side contract and truthful 'PDF generation unavailable' state; produce exact Codex API requirements. Do not duplicate the PDF engine in GUI or generate dummy PDFs while claiming genuine export. Native save-file dialog, explicit filename and location, allowed extension checks, overwrite confirmation, temporary file handling, atomic finalization when supported, accurate success/failure, bounded file operation, and cleanup of partial output. Verify actual PDF bytes/pages/text and actual saved path in a controlled test; do not claim success on button click alone.

STEP 06 - PATIENT SESSION EXPORT
Implement an explicit, user-initiated 'Export this examination' workflow, separated from exporting all patient history. Show exactly what will be included: examination metadata, original image (optional), genuine screening results, missing-stage status, source/version information and audit metadata. No accidental inclusion of a different patient/exam, no new plaintext clinical database, no cloud transfer, no sensitive absolute paths or raw patient details in ordinary logs. Avoid implying exported data is encrypted unless the format truly is. If secure export backend is absent, provide safe front-end review and detailed backend handoff rather than inventing encrypted export.

STEP 07 - REAL AUDIT LEDGER UI
Inspect whether backend exposes persisted, tamper-aware audit entries. Add a read-only GUI viewer with timestamp, event type, request ID and status; filter by active examination; never fake an audit trail. Audit 'report requested', 'export completed', 'export failed' only when the backend successfully persists them. If no API exists, display 'Audit history unavailable' and document the minimal typed API required. Do not make HIPAA/FIPS or other certification claims.

STEP 08 - NONBLOCKING LIFECYCLE
Separate idle, preparing, generating, save-requested, saved, failed and cancelled export states. Use bounded channels or existing concurrency; do not block egui frames. Track stable request IDs, reject stale events, preserve report data across recoverable failures, and prevent duplicate exports. Cancellation must not claim that a running operation stopped unless acknowledged. Image model inference remains separate from PDF generation and normal chat.

STEP 09 - PRIVACY AND CLINICAL TRUTHFULNESS
Stage I is a routing/screening result, not a final DR diagnosis. Stage II lesion/vessel masks, Stage III grade, and explainability must be marked unavailable until actual artifacts exist. Do not generate false clinical statements, severity, lesion counts or attribution maps. Add a non-diagnostic disclaimer in both preview and actual PDF when available. No automatic third-party transfers, unsafe path interpolation or patient-level cross-contamination.

STEP 10 - TEST AND VISUAL VERIFY
From rust/ execute cargo fmt --all --check, cargo clippy --all-targets -- -D warnings, cargo test, cargo build --release, git diff --check. Add tests for report source association, missing stage flags, actual Stage I values, no fabricated DR grades, model checksum metadata, save dialog cancel, invalid output path, duplicate requests, export failure, stale event and examination switching, patient session inclusion scope, ledger unavailable/readiness, and regression of existing 81 tests. For genuine PDF export, verify a real generated PDF with a PDF reader and check pages and key text. Launch native desktop and capture screenshots only of genuine states: overview, report preview, export dialog or ready state, saved result, missing backend, small window and audit ledger. Do not fabricate an 'export succeeded'

screenshot.

STEP 11 - DOCUMENT AND HANDOFF
Record actual tests and exit codes, screenshots, all changed file paths/functions/rationale, available backend APIs, precise missing Codex functions, branch/dependency packaging risk (hard-coded ../../Dird path), memory changes, remaining blockers and Task06 handoff. Windows runtime behavior remains unverified unless actually executed. No fake completion percentage.

DEFINITION OF DONE
Report page displays true Stage I data, original image and truthful missing-stage states; user can review report, explicit local PDF generation is wired to actual backend if available, actual saved output validated; session export does not cross examinations; real ledger shown only when backend supplies events; GUI remains nonblocking; tests/build/desktop checks pass; docs and memory updated; no backend modifications. If a live PDF backend or audit API is missing, declare Task05 PARTIAL and give Codex specific minimal requirements. Do not start Task06.

FINAL RESPONSE FORMAT
A. Initial audit and Git status; B. Report UX implementation; C. Actual PDF backend integration; D. Session export and privacy; E. Audit ledger and record authenticity; F. Async/cancel/error handling; G. Test counts and exact commands; H. Real screenshots; I. File-by-file changes; J. Docs and memory; K. Missing Codex APIs; L. Task05 verdict COMPLETE/PARTIAL/BLOCKED; M. Task06 handoff.

EXECUTION INSTRUCTION
ANTIGRAVITY: START GUI TASK 05 NOW. Audit the real repositories, implement actual native Rust report presentation and safe user-initiated export using only verified Codex public services, compile, test, launch, verify actual documents and screenshots, then document every modification. NEVER edit Codex-owned backend, never invent DR grades, and never claim PDF success without a real artifact. Proceed through all implementable scoped work in this task.

# APPENDIX B / FULL JSON MASTER PROMPT

Complete valid JSON prompt, printed as searchable selectable text. An identical .json file accompanies the PDF.


```
{
  "prompt_id": "RETINACARE-GUI-P01-TASK05",
  "project": "RetinaCare",
  "agent": "Antigravity",
  "phase": "GUI Phase 1",
  "task": "05",
  "title": "Clinical Report Presentation, Local PDF Export, Patient Session Export and Audit
      Ledger UI",
  "priority": "P0 - Hackathon-critical",
  "mode": "AUDIT_RESEARCH_PLAN_IMPLEMENT_TEST_VERIFY_DOCUMENT",
  "role": [
    "Principal Native Rust GUI Engineer",
    "Medical Report UX Architect",
    "Secure Export Integration Engineer",
    "Typed Rust Service Engineer",
    "QA and Privacy Engineer"
  ],
  "mission": {
    "goal": "Make RetinaCare's existing medical screening findings reviewable and exportable
        through a real native Rust GUI and a genuinely callable Codex PDF/report service where
        available.",
    "implementation": "Actual Rust coding, tests, native launch, artifact verification,
        documentation; do not stop at planning.",
    "critical": "Stage I screening is not clinical DR severity; never fabricate lesion masks, DR
        grade, Grad-CAM, audit entries or PDF success."
  },
  "repos": {
    "gui_expected": "/home/claude/Desktop/Hackthon dezine/rust",
    "backend_read_only": "/home/claude/Desktop/Dird/native/crates",
    "verify_paths_and_branches": true,
    "reported_gui_branch": "dezine",
    "reported_backend_branch": "main",
    "no_force_merge": true,
    "preserve_dirty_worktrees": true
  },
  "previous_task04_report": {
    "reported_gui_tests": 81,
    "reported_features": [
      "retinacare-app / retinacare-ai-bridge path dependencies",
      "MedicalInferenceService and LiveCodexBackendService",
      "Nonblocking Stage I request-event-result",
      "Truthful unavailable Stage II and Stage III states",
      "Stage I screening result UI"
    ],
    "important": "Reports must be verified against code and runtime before reliance; reported ≠
        independently verified.",
    "source": "User-provided GUI Task04 report"
  },
  "truthful_model_stage_policy": {
    "official_medical_stages": 3,
    "stage1": "Screening/routing only; show real score and original threshold semantics without
        unsupported clinical calibration claims.",
    "stage2": "No lesion/vessel artifacts until authentic results exist.",
    "stage3": "No DR Grade 0-4 until authentic grader runs.",
    "explainability": "No Grad-CAM unless model attribution computed.",
    "pdf": "Application report/export workflow, not fourth medical AI stage.",
    "chat": "General conversational LLM remains independent."
  },
  "ownership": {
    "gui_owned": [
      "rust/src/",
      "rust/Cargo.toml when necessary",
      "docs/features/GUI-P05-clinical-report-export/",
      "GUI-relevant memory entries"
    ],
    "backend_read_only": [
      "native/crates/",
      "ai_engine/",
      "models/",
      "backend PDF engine",
      "SQLCipher persistence"
    ],
```

```
  "forbidden": [
    "Modify Codex-owned files",
    "Create duplicate inference or PDF engine",
    "Add HTML/CSS/JS/WebView/React/Electron/Tauri",
    "Push, commit or force merge without approval"
  ]
},
"mandatory_skills": {
  "ponytail": "Reuse existing installation; minimal complete solution.",
  "superpowers": "Use relevant research, planning, TDD, debugging and verification skills.",
  "AGENTS_md": true,
  "avoid_duplicate_clones": true
},
"step01_audit": {
  "tasks": [
    "Read AGENTS.md and memory index/active phase",
    "Read Task04 handoff and actual rust/src/ GUI",
    "Inspect rust/Cargo.toml and path dependencies",
    "Inspect backend retinacare-app/reporting and retinacare-reports public APIs read-only",
    "Inspect encryption and audit APIs read-only",
    "Check branch/status/working trees",
    "Run existing tests before changes",
    "Record real Stage I and missing Stage II/III availability"
  ],
  "deliverable": "Verified audit, API inventory, ownership plan"
},
"step02_docs": {
  "directory": "docs/features/GUI-P05-clinical-report-export/",
  "files": [
    "README.md",
    "01-audit.md",
    "02-research.md",
    "03-requirements.md",
    "04-implementation-plan.md",
    "05-report-data-contract.md",
    "06-export-state-machine.md",
    "07-audit-contract.md",
    "08-tests.md",
    "09-test-results.md",
    "10-desktop-verification.md",
    "11-file-change-report.md",
    "12-limitations-and-backend-gaps.md",
    "13-task06-handoff.md"
  ],
  "requirement": "Factual, concise, Obsidian-compatible; no filler."
},
"step03_report_contract": {
  "reuse_first": [
    "RetinalExaminationView",
    "Stage1ScreeningResult",
    "BackendReadiness",
    "Exam/Patient/Request IDs",
    "artifact availability"
  ],
  "include_when_source_available": [
    "Original image and laterality",
    "Actual Stage I score, logit, threshold and route",
    "model ID/version/hash/provider",
    "Exam/request correlation",
    "Timestamp",
    "Research use limitation"
  ],
  "hard_checks": [
    "Cross-patient isolation",
    "No stale results",
    "No Stage III DR severity unless real",
    "Never call raw sigmoid clinically calibrated risk without valid evidence",
    "No invented attribution or masks"
  ]
},
"step04_report_gui": {
  "required": [
    "Readable report summary",
    "Original fundus thumbnail with correct aspect ratio",
    "Stage I-only result card",
    "Model provenance and artifact statuses",
    "Preview / save UX",
    "Responsive small-window scroll",
    "Dark existing egui/eframe theme",
```

```
      "Loading and error states"
    ],
    "prohibited": [
      "Redesign Home/Chat",
      "Fake severity or clinical scores",
      "Heavy blocking work on GUI thread"
    ]
  },
  "step05_pdf_export": {
    "audit_real_api_before_coding": true,
    "prefer": "Call existing Rust PDF service from GUI adapter; do not duplicate engine.",
    "required": [
      "Preview backed by actual data",
      "Native save dialog",
      "Explicit destination and confirmation",
      "Correct .pdf extension and filename validation",
      "Safe overwrite handling",
      "Background generation",
      "Temporary output cleanup",
      "Actual file/page/text validation in integration test",
      "Correct success only after confirmed write",
      "Non-diagnostic disclaimer inside PDF"
    ],
    "when_api_missing": "Implement only a testable UI contract and truthful unavailable state,
        document minimal Codex API with method signatures; Task05 remains PARTIAL.",
    "security": [
      "No unencrypted new database",
      "No raw patient data in logs",
      "No silent external upload",
      "No unsafe path concatenation"
    ]
  },
  "step06_session_export": {
    "goal": "Explicit per-examination export with selected fields and optional image, not all
        patients.",
    "required": [
      "Scope preview",
      "Patient/exam ID verification",
      "Opt-in inclusion list",
      "Original image inclusion toggle",
      "Real model provenance",
      "Sensitive data omission",
      "Clear encryption status; no unsupported guarantee",
      "Correct cancellation and retry"
    ],
    "backend_missing": "Document missing export API and leave sensitive export unavailable rather
        than write unauthorized plaintext records."
  },
  "step07_audit_ledger": {
    "goal": "Read-only presentation of genuinely persisted backend audit events.",
    "fields": [
      "event time",
      "event type",
      "examination ID",
      "request ID",
      "status",
      "non-sensitive metadata"
    ],
    "rules": [
      "No fake events",
      "No implied HIPAA/FIPS certification",
      "Show unavailable if no persisted audit API",
      "Only claim successful audit write after backend acknowledges persistence"
    ],
    "backend_handoff": "Exact minimal ledger list/filter and export-event append APIs."
  },
  "step08_async_state": {
    "states": [
      "Idle",
      "Preparing",
      "Generating",
      "SaveRequested",
      "Saved",
      "Failed",
      "Cancelled",
      "BackendUnavailable"
    ],
    "required": [
      "Stable IDs",
```

```
      "No duplicate requests",
      "Bounded channel or existing queue",
      "Nonblocking egui update",
      "No false cancellation acknowledgement",
      "Stale result rejection",
      "UI remains responsive"
    ]
  },
  "step09_testing": {
    "working_dir": "rust/",
    "commands": [
      "cargo fmt --all --check",
      "cargo clippy --all-targets -- -D warnings",
      "cargo test",
      "cargo build --release",
      "git diff --check"
    ],
    "tests": [
      "Actual Stage I data displayed faithfully",
      "Stage II/III unavailable and no grade fabricated",
      "Correct report/exam association",
      "No cross-patient export",
      "PDF generation service success/failure",
      "Actual PDF artifact validation when supported",
      "Save dialog cancel",
      "Invalid destination",
      "Duplicate export prevention",
      "Stale event rejection",
      "Cancelled export state",
      "Ledger missing API state",
      "Ledger filtering genuine records",
      "GUI Home/Chat/viewer regressions"
    ],
    "live_verification": [
      "Native Ubuntu GUI launch",
      "Real report overview",
      "Real PDF save and inspect when backend available",
      "Missing backend and failed save",
      "Small-window layout",
      "Correct disclaimer",
      "No synthetic success screenshots"
    ]
  },
  "step10_reporting": {
    "docs_update": [
      "all Task05 docs",
      "memory/00-index.md",
      "memory/active-phase.md",
      "memory/decisions.md",
      "memory/gotchas.md"
    ],
    "file_change_report_fields": [
      "Exact path",
      "Created/modified",
      "Functions and structs",
      "Why necessary",
      "Code behavior",
      "Dependencies",
      "Tests",
      "Evidence",
      "Limitations"
    ],
    "coordination": "Do not edit Codex backend; specify missing backend contracts precisely;
        document portability risk of relative ../../Dird path."
  },
  "definition_of_done": {
    "requirements": [
      "Source-grounded report screen",
      "Real Stage I values and proper caveat",
      "Accurate missing-stage states",
      "Explicit user-initiated local PDF export when service available",
      "Actual PDF validated",
      "Session export scoped to correct examination",
      "Actual persisted ledger or truthfully unavailable",
      "Async, errors, cancellation and stale events tested",
      "Regression checks pass",
      "Desktop screenshots from real app",
      "Docs, memory, and change report completed",
      "No backend modifications"
```

```
    ],
    "if_missing_api": "Report PARTIAL with blocker and exact Codex handoff; do not pretend
        COMPLETE.",
    "next_task": "Task06 - Patient and Examination Management; handoff only, do not begin."
  },
  "final_report_sections": [
    "A. Audit and Git safety",
    "B. Report UI",
    "C. PDF backend service integration",
    "D. Session export/privacy",
    "E. Real audit ledger",
    "F. Nonblocking lifecycle",
    "G. Tests/build",
    "H. Desktop and PDF verification",
    "I. Exact file changes",
    "J. Documents/memory",
    "K. Missing Codex APIs",
    "L. Status COMPLETE/PARTIAL/BLOCKED",
    "M. Task06 handoff"
  ],
  "execute": "ANTIGRAVITY: Continue from Task04; implement GUI Task05 now. Audit the actual
      backend PDF/audit APIs, write native Rust report and export UX, use genuine Stage I values,
      preserve unavailable Stage II/III, verify PDF bytes only on real success, run tests and
      screenshots, update docs/memory. No GUI redesign outside Task05, no backend edits, no fake
      diagnoses or audit records, no automatic Git push."
}
```